UndefinedBehaviorSanitizer (UBSan) Beginner's Guide

Catch undefined behavior in C/C++ with one compiler flag

1. What UBSan does

Some C/C++ operations have no defined result, such as signed integer overflow, dividing by zero, or shifting by too many bits. This is called **undefined behavior (UB)**. The compiler may assume UB never happens, so the program can print garbage, behave differently at `-O2` than at `-O0`, or work until you change compilers. UBSan inserts checks for these operations and prints a message with the file, line and column when one happens.

ASan catches memory bugs. UBSan catches rule-breaking in the language itself. They complement each other.

2. What it catches

- Signed integer overflow
- Integer division by zero
- Shifts that are too large or negative
- Null pointer dereference and member access
- Array index out of bounds (arrays with known size)
- Misaligned pointers
- Falling off the end of a function that should return a value
- Invalid `bool` / `enum` values and bad downcasts (`vptr`, needs RTTI)

It does **not** flag unsigned overflow, because that is well defined in C++ (it wraps around).

3. Compile

```
g++ -fsanitize=undefined -g -O0 main.cpp -o main
```

- `-fsanitize=undefined`: enables the whole group of UB checks
- `-g`: file and line info in messages
- `-O0`: no optimization, so the bug is not folded away while you learn

By default UBSan **prints the error and keeps running**. To stop at the first error:

```
g++ -fsanitize=undefined -fno-sanitize-recover=undefined -g -O0 main.cpp -o main
```

4. Practice program

Save as `main.cpp`. You choose the case and a number from the command line, so the compiler cannot spot the bug at compile time.

```
#include <climits>
#include <cstdlib>
#include <iostream>

int overflow(int n) { int x = INT_MAX; return x + n; }
int divzero(int n)  { return 10 / n; }
int shift(int n)    { return 1 << n; }
int oob(int n)      { int a[3] = {1, 2, 3}; return a[n]; }
void nullp()        { int* p = nullptr; *p = 5; }
int noreturn(int n) { if (n > 100) return 1; }   // missing return

int main(int argc, char** argv) {
    int which = (argc > 1) ? std::atoi(argv[1]) : 1;
    int n     = (argc > 2) ? std::atoi(argv[2]) : 1;
    switch (which) {
        case 1: std::cout << overflow(n) << "\n"; break;
        case 2: std::cout << divzero(n)  << "\n"; break;
        case 3: std::cout << shift(n)    << "\n"; break;
        case 4: std::cout << oob(n)      << "\n"; break;
        case 5: nullp();                         break;
        case 6: std::cout << noreturn(n) << "\n"; break;
    }
    return 0;
}
```

5. Run each case and read the output

```
./main 1 1     # signed overflow
./main 2 0     # division by zero
./main 3 40    # shift too large
./main 4 5     # index out of bounds
./main 5 0     # null pointer
./main 6 1     # missing return
```

Messages look like this (exact wording varies by compiler):

```
main.cpp:5:48: runtime error: signed integer overflow:
    2147483647 + 1 cannot be represented in type 'int'
main.cpp:6:44: runtime error: division by zero
main.cpp:7:44: runtime error: shift exponent 40 is too large for 32-bit type 'int'
main.cpp:8:62: runtime error: index 5 out of bounds for type 'int [3]'
main.cpp:9:41: runtime error: store to null pointer of type 'int'
runtime error: execution reached the end of a value-returning function
    without returning a value
```

How to read one:

- `main.cpp:5:48`: file, **line 5, column 48**, the exact expression
- `runtime error:`: every UBSan message starts with this
- The rest says which rule was broken and shows the values involved (`2147483647 + 1`)

Case 5 prints the UBSan message first, then crashes with a segfault. In cases 1 to 4 and 6 the program continues and prints a meaningless result, unless you built with `-fno-sanitize-recover`.

6. What each case looks like

Run	Bug	Message starts with
`./main 1 1`	Signed overflow	`signed integer overflow: 2147483647 + 1 ...`
`./main 2 0`	Division by zero	`division by zero`
`./main 3 40`	Bad shift	`shift exponent 40 is too large ...`
`./main 4 5`	Out-of-bounds index	`index 5 out of bounds for type 'int [3]'`
`./main 5 0`	Null pointer write	`store to null pointer of type 'int'`
`./main 6 1`	Missing return	`execution reached the end of a value-returning function ...`

7. Useful options

```
UBSAN_OPTIONS=print_stacktrace=1 ./main 1 1   # add a call stack to each report
UBSAN_OPTIONS=halt_on_error=1 ./main 1 1     # stop at first error (best with -fno-sanitize-recover)
```

Enable or disable individual checks:

```
-fsanitize=signed-integer-overflow,shift,null,bounds
-fno-sanitize=alignment
```

Combine with ASan (a common debug build):

```
g++ -fsanitize=address,undefined -g -O1 -fno-omit-frame-pointer main.cpp -o main
```

Do not add `thread`; ThreadSanitizer cannot be combined with ASan.

8. Using UBSan in projects

Makefile

```
CXXFLAGS = -std=c++17 -Wall -Wextra -g -O1 -fsanitize=undefined
LDFLAGS  = -fsanitize=undefined
```

CMake

```
add_compile_options(-fsanitize=undefined -g)
add_link_options(-fsanitize=undefined)
```

VS Code (`tasks.json`): add `"-fsanitize=undefined"` to the `args` list.

9. Limits and gotchas

- Only checks code paths that actually run
- Some overhead, usually less than ASan
- Does not find memory errors like heap overflow or use-after-free (ASan) or uninitialized reads (Valgrind / MemorySanitizer)
- Unsigned wraparound and narrowing conversions are not reported by default
- Messages and checks can differ between GCC and Clang, and between versions
- Build everything with the same flags, including the link step

10. Cheat sheet

What	Command
Build with UBSan	`g++ -fsanitize=undefined -g -O0 main.cpp -o main`
Stop at first error	add `-fno-sanitize-recover=undefined`
Show stack traces	`UBSAN_OPTIONS=print_stacktrace=1 ./main`
ASan + UBSan together	`-fsanitize=address,undefined`

Reading order: **"runtime error" line**, then **file:line:column**, then the **values** in the message.

11. Practice tasks

1. Run all six cases and note the message and line:column for each.
2. Fix case 1 (use `long long`, or check `x > INT_MAX - n` first) and confirm the report disappears.
3. Try `./main 3 31` (valid) and `./main 3 32` (error) to see exactly where the shift rule breaks.
4. Rebuild with `-fno-sanitize-recover=undefined` and watch the program stop at the first error.

12. Troubleshooting

- **No report at all:** the code path did not run, or the compiler removed it. Rebuild with `-O0`.
- **No line numbers:** forgot `-g`.
- **Link errors about `__ubsan_...`:** add `-fsanitize=undefined` to the link step too.
- **Too much output:** use `-fno-sanitize-recover=undefined` to stop at the first error.
- **Next topic:** Valgrind.